Aluno: Eduardo Magalhães

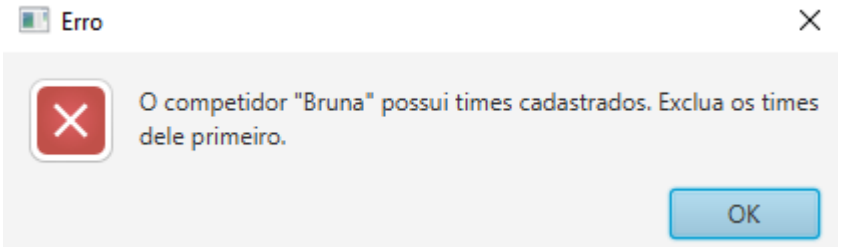
Sistema: Poketeams

O Trabalho 1 seguia MVC com DAO, mas as telas faziam muito mais do que exibir dados:

- Os controllers chamavam os DAOs diretamente (IncTimeController usava PokemonDAO e TimeDAO).
- As regras de negócio ficavam nos controllers: validar nomes, checar se o pokémon existe, bloquear repetidos e exigir ao menos 1 pokémon no time.
- Havia código duplicado: o método listar em quase todos os controllers.
- O visaoController concentrava um switch/if por categoria (Geral, Competidor, Time, Pokémon) em três métodos.
- As telas carregavam dados só ao abrir, e a operação de cadastro não deixava rastro nem podia ser desfeita.

Item	**O que apresentar**
Identificação	ServiceLayer **Finalidade:** concentrar as regras de negócio e isolar a interface do acesso a dados (Controller → Service → DAO). **Classes/pacotes:** pacote service: CompetidorService, PokemonService, TimeService.
Situação Anterior	Controllers validavam dados, interpretavam erros SQL e chamavam o DAO. A regra de montar um time estava duplicada entre Inclusão e Atualização, e trocar o banco exigiria alterar as telas.
Solução implementada	Cada entidade ganhou um service com responsabilidade própria: validação e orquestração. Os controllers só repassam o que o usuário digitou e exibem o resultado. O TimeService reaproveita o PokemonService para resolver nomes em IDs.
Justificativa e verificação	**Benefício:** controllers mais enxutos, regras em um só lugar, mensagens de erro claras e nenhum import java.sql na camada de interface.

	Evidência: os controllers de Inclusão, Exclusão e Atualização usam service.cadastrar/atualizar/excluir/listar. Ao tentar excluir um competidor com times, a tela mostra a mensagem do service “O competidor “Fulano” possui times cadastrados. Exclua os times dele primeiro”




```
public void excluir(Competidor competidor) {
    if (competidor == null) {
        throw new IllegalStateException("Selecione o competidor a
excluir.");
    }
    try {
        dao.excluir(competidor.getCompetidorId());
    } catch (SQLException e) {
        if (FK_VIOLADA.equals(e.getSQLState())) {
            throw new IllegalStateException(
                    "O competidor \"" + competidor.getNome()
                            + "\" possui times cadastrados. Exclua
os times dele primeiro.", e);
        }
        throw new IllegalStateException("Erro ao excluir
competidor: " + e.getMessage(), e);
    }
    notificar();
}
```

Item	O que apresentar
Identificação	Strategy **Finalidade:** permitir que a tela de busca troque o comportamento de cada categoria sem alterar o controller. **Classes/pacotes:** pacote strategy, EstrategiaBusca, BuscaGeral, BuscaCompetidor, BuscaTime, BuscaPokemon. Contexto: visaoController.
Situação Anterior	O visaoController decidia colunas visíveis, filtro e remoção de repetidos com if/switch por categoria. Criar uma categoria nova obrigava a editar vários métodos. Além disso, a categoria Pokémon não mostrava pokémons fora de times, porque os dados vinham só do resumo dos times.
Solução implementada	Cada categoria virou uma classe que implementa EstrategiaBusca. O controller guarda a estratégia selecionada no ComboBox e delega: define quais colunas aparecem e chama buscar(). Cada estratégia busca seus dados pelo service apropriado (BuscaPokemon usa o PokemonService).
Justificativa e verificação	**Benefício:** nova categoria = nova classe + uma linha no addAll, sem tocar nas existentes (aberto/fechado). O controller perdeu ajustarColunas, ignorar e combina. **Evidência:** ao alternar o ComboBox da tela Visão, as colunas e linhas mudam; na categoria Pokémon, aparecem também pokémons que não estão em nenhum time.

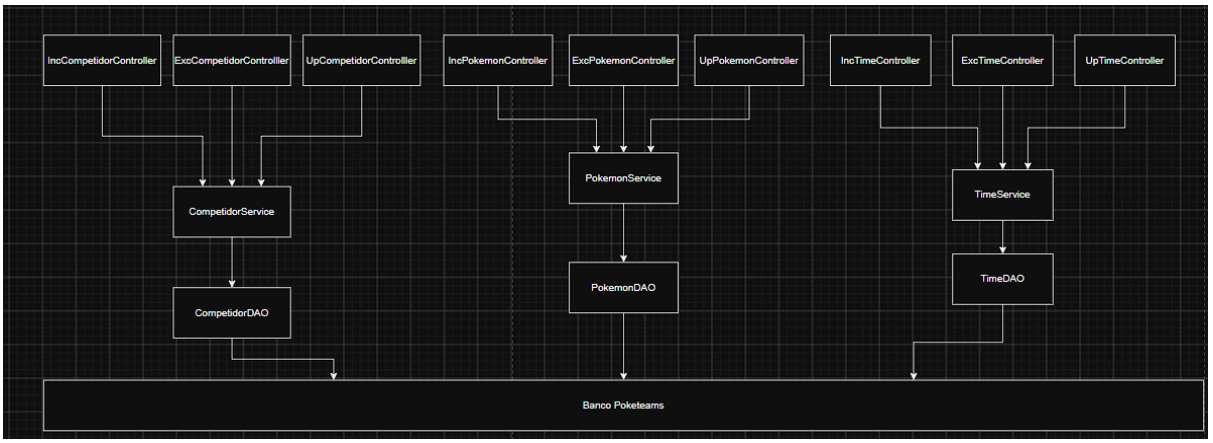


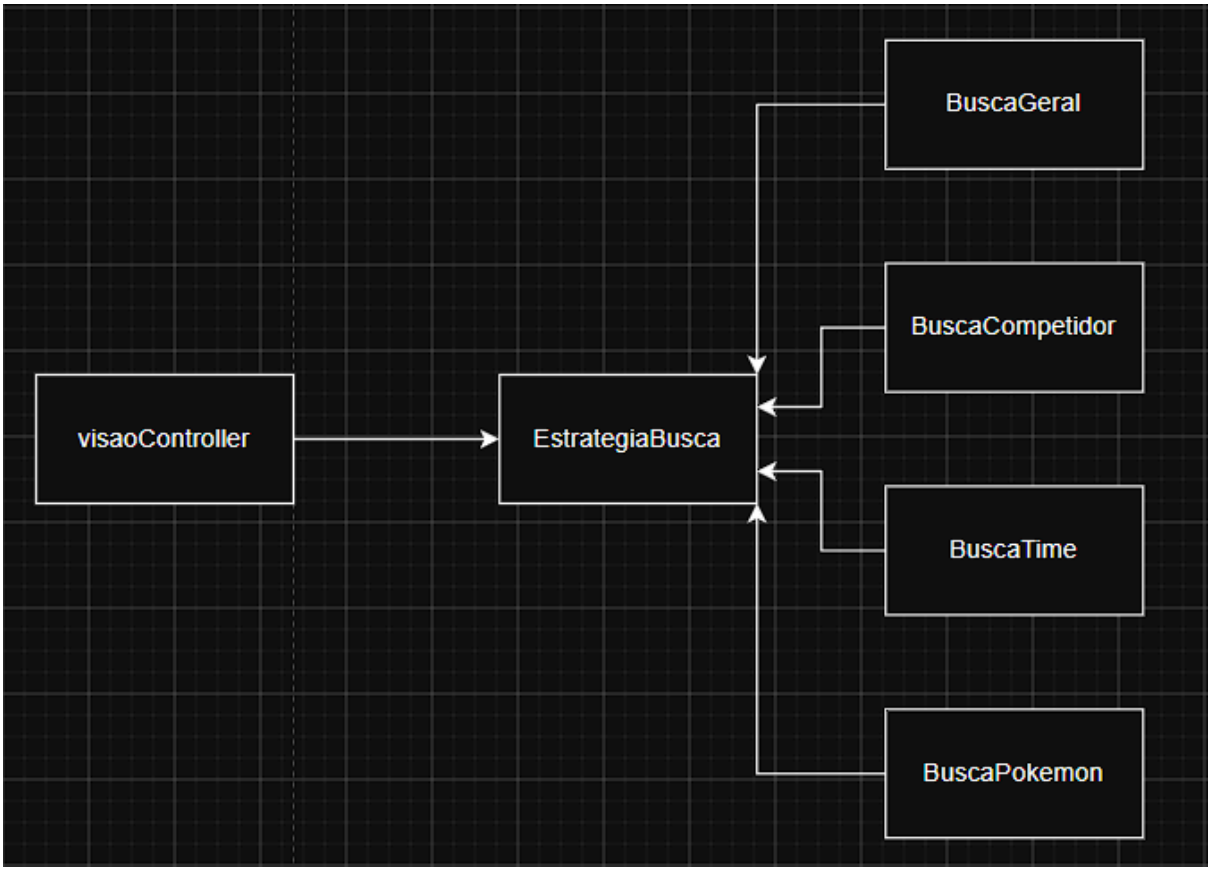
Item	O que apresentar
Identificação	Observer **Finalidade:** avisar automaticamente as telas interessadas quando os dados de competidores mudam. **Classes/pacotes:** observer.CompetidorObserver, service.CompetidorService (sujeito), IncCompetidorController (observador).
Situação Anterior	Caso um usuario precisasse cadastrar muitos competidores de uma vez poderia ter problemas para ter noção de quantos ja cadastrou.
Solução implementada	Um contador de quantos competidores foram cadastrados
Justificativa e verificação	**Benefício:** Melhor controle de quantos competidores já foram cadastrados.

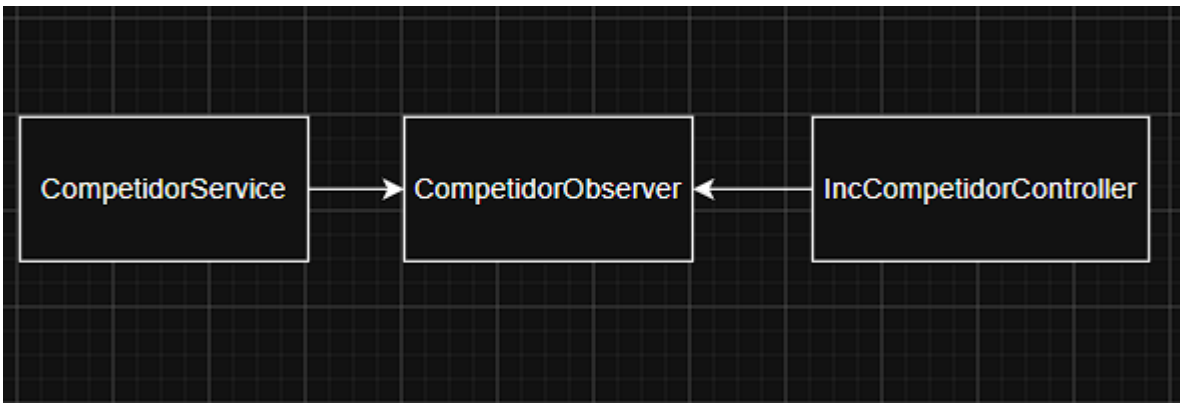


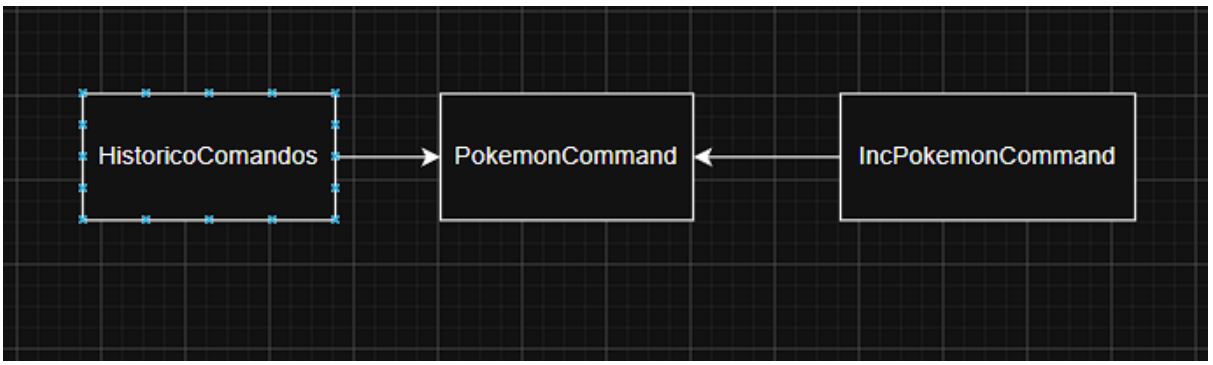
Item	O que apresentar
Identificação	Command **Finalidade:** representar o cadastro de pokémon como objeto, permitindo registrar o histórico e desfazer a operação. **Classes/pacotes:** pacote command: Command, IncluirPokemonCommand, HistoricoComandos. Cliente: IncPokemonController. Receiver: PokemonService.
Situação Anterior	O controller chamava o cadastro diretamente, a operação não deixava registro e não havia como desfazer um cadastro feito por engano.
Solução implementada	O controller cria um IncluirPokemonCommand com os dados e o entrega ao HistoricoComandos (Invoker), que o executa e empilha. O comando guarda o pokémon criado e, em desfazer(), o exclui pelo service. O botão “Desfazer último” desfaz na ordem inversa.
Justificativa e verificação	**Benefício:** a operação vira um objeto com dados e comportamento de desfazer, com histórico em pilha e controller desacoplado do cadastro.

	Evidência: após cadastrar um pokémon, o botão “Desfazer último” fica ativo; ao clicar, o registro some do banco






CompetidorService
CompetidorObserver
IncCompetidorController


HistoricoComandos
PokemonCommand
IncPokemonCommand